Python

- History and Overview
- Installing Python and Setting up IDEs (e.g., VS Code, PyCharm, Jupyter Notebook)
- Writing and Running Your First Python Program
- Python Syntax and Indentation
- Python Comments

Basic Python Concepts

1. **Variables and Data Types**
    - Numbers (int, float, complex)
    - Strings
    - Booleans
    - NoneType
2. **Operators**
    - Arithmetic, Assignment, Comparison
    - Logical, Bitwise, Membership, Identity
3. **Input and Output**
    - Input from User
    - Printing Outputs
    - String Formatting (f-strings, format(), % operator)
4. **Control Flow**
    - If-Else Statements
    - Loops: For and While
    - Break, Continue, Pass

Intermediate Python Concepts

1. **Data Structures**
    - Lists: Creation, Methods, Comprehensions
    - Tuples: Immutable Sequences
    - Sets: Unique Elements and Operations
    - Dictionaries: Key-Value Pairs, Comprehensions
2. **Functions**

    - Defining Functions
    - Arguments: Positional, Keyword, Default, Arbitrary
    - Return Statement
    - Lambda Functions
    - Recursion
    - Decorators
3. **Modules and Packages**

    - Importing Built-in Modules
    - Creating and Importing Custom Modules
    - Using `pip` to Install External Libraries
4. **File Handling**

    - Reading and Writing Text Files
    - Working with CSV and JSON
    - Exception Handling in File Operations

Advanced Python Concepts

1. **Object-Oriented Programming (OOP)**

    - Classes and Objects
    - Inheritance
    - Polymorphism
    - Encapsulation
    - Magic/Dunder Methods (e.g., `__init__`, `__str__`, `__repr__`)
2. **Error and Exception Handling**

    - Try, Except, Else, Finally
    - Custom Exceptions
3. **Iterators and Generators**

    - Creating and Using Iterators
    - Using `yield` in Generators
4. **Comprehensions**

    - Nested Comprehensions
    - Set and Dictionary Comprehensions

Specialized Topics

1. **Regular Expressions**

    - Pattern Matching
    - Search, Match, and Replace
2. **Multithreading and Multiprocessing**
    - Threading Module
    - Global Interpreter Lock (GIL)
    - Multiprocessing Module
3. **Networking**
    - Sockets
    - HTTP Requests with `requests` Library
4. **Working with Databases**
    - SQLite and `sqlite3` Module
    - CRUD Operations

Data Science and Visualization

1. **NumPy**: Arrays and Linear Algebra Operations
2. **Pandas**: DataFrames, Series, Data Cleaning
3. **Matplotlib**: Basic and Advanced Plotting
4. **Seaborn**: Statistical Visualization

Web Development

1. **Flask/Django**: Creating Web Applications
2. **APIs**: Building and Consuming REST APIs
3. **Web Scraping**: Using `BeautifulSoup` and `Scrapy`

Testing and Debugging

1. **Unit Testing**: Using `unittest`
2. **Debugging Tools**: `pdb` and IDE Debugging Features
3. **Code Profiling**: Using `cProfile`

Advanced Topics

1. **Machine Learning**
    - Using `scikit-learn`
    - Basic ML Models (Linear Regression, Classification)

2. **Automation**
    - Automating Tasks with `selenium` and `pyautogui`
3. **Packaging and Deployment**
    - Creating Python Packages
    - Virtual Environments and Dependency Management

Neural Networks

1.1 Basics of Neural Networks

- **What is a Neural Network?**: Biological inspiration, artificial neurons, and neural layers.
- **Components of a Neural Network**:
    - **Neurons (Units)**: Artificial neurons, activation functions.
    - **Weights and Biases**: How they affect the learning process.
    - **Architecture**: Input layer, hidden layers, output layer.
    - **Types of Neural Networks**: Feedforward Neural Networks, Recurrent Neural Networks, Convolutional Neural Networks, etc.

1.2 Neural Network Terminology

- **Activation Functions**: Sigmoid, Tanh, ReLU, Leaky ReLU, Softmax.
- **Forward Propagation**: Process of computing outputs from inputs through layers.
- **Backward Propagation**: The method of updating weights using the chain rule of derivatives (Gradient Descent).
- **Loss Function**: Cross-entropy, Mean Squared Error, etc.
- **Optimizer Algorithms**: Stochastic Gradient Descent (SGD), Adam, RMSProp, etc.

1.3 Mathematical Foundations

- **Linear Algebra for Neural Networks**:
    - Vectors, matrices, and operations (dot product, transpose).
- **Calculus for Neural Networks**:
    - Derivatives, partial derivatives, and chain rule.
- **Probability and Statistics**:
    - Probability theory for understanding loss functions and predictions.

---

2. Building Simple Neural Networks

2.1 Single-Layer Perceptron (SLP)

- **Definition and Structure**: Understanding single-layer neural networks.
- **Mathematical Formulation**: Weighted sums, activation functions.
- **Training Neural Networks**:
    - The concept of forward pass and backward pass.
    - Gradient Descent algorithm for weight updates.

2.2 Multi-Layer Perceptron (MLP)

- **Architecture of MLP**: Multiple layers and how they work.
- **Activation Functions**: ReLU, Sigmoid, Tanh.
- **Backpropagation**: Deriving the error, applying the chain rule, and updating weights.
- **Training MLP**: Epochs, batch processing, learning rate.

---

3. Optimization and Learning Techniques

3.1 Gradient Descent and Variants

- **Gradient Descent (GD)**: Concept, update rule, and convergence.
- **Stochastic Gradient Descent (SGD)**: Mini-batch gradient descent and advantages over traditional GD.
- **Momentum**: Adding momentum to accelerate learning.
- **Learning Rate Scheduling**: Constant, decaying, and adaptive learning rates.

3.2 Regularization Techniques

- **Overfitting and Underfitting**: The bias-variance tradeoff.
- **L1 and L2 Regularization**: Adding penalty terms to the loss function.
- **Dropout**: Randomly disabling neurons to prevent overfitting during training.
- **Early Stopping**: Stopping training once the validation error starts to increase.

3.3 Advanced Optimization Algorithms

- **Adam Optimizer**: Adaptive moment estimation (combining momentum and RMSProp).
- **RMSProp**: Root mean square propagation for faster convergence.
- **Adagrad, Adadelta**: Adaptive learning rate methods.

---

4. Advanced Neural Network Architectures

4.1 Convolutional Neural Networks (CNNs)

- **Understanding CNNs**: The need for CNNs in image recognition tasks.
- **Layers in CNN**:
    - **Convolutional Layer**: Applying filters to extract features.
    - **Pooling Layer**: Max pooling, average pooling for dimensionality reduction.
    - **Fully Connected Layer**: Flattening after convolution and pooling.
- **Architecture**: LeNet, AlexNet, VGG, and ResNet.
- **Applications of CNNs**: Image classification, object detection, face recognition.

4.2 Recurrent Neural Networks (RNNs)

- **Understanding RNNs**: The concept of memory in sequences.
- **Vanishing Gradient Problem**: Challenges in training RNNs.
- **Long Short-Term Memory (LSTM)**: The architecture of LSTMs to address the vanishing gradient issue.
- **Gated Recurrent Units (GRUs)**: A simpler alternative to LSTM.
- **Applications of RNNs**: Time series forecasting, speech recognition, text generation.

4.3 Generative Models

- **Autoencoders**: Understanding how autoencoders can learn compressed representations of data.
    - **Applications**: Denoising autoencoders, anomaly detection.
- **Generative Adversarial Networks (GANs)**:
    - **Architecture**: Generator and Discriminator.
    - **Training Process**: Adversarial training and loss functions.
    - **Applications of GANs**: Image generation, deepfake creation, style transfer.

4.4 Transformer Networks

- **Attention Mechanism**: Understanding attention as a method to focus on important features in the input sequence.
- **Self-Attention**: How each word/element in the sequence attends to others.
- **Transformer Architecture**: Encoder-decoder structure, multi-head attention, position encoding.
- **Applications of Transformers**: Natural language processing tasks like machine translation (BERT, GPT).

---

5. Neural Networks for Natural Language Processing (NLP)

5.1 Word Embeddings

- **Word2Vec**: Continuous bag of words (CBOW), skip-gram models.
- **GloVe (Global Vectors for Word Representation)**: Learning word embeddings based on co-occurrence matrix.
- **FastText**: Embeddings based on sub-word information.

5.2 Recurrent Networks in NLP

- **RNNs in NLP**: Language modeling and text generation.
- **LSTM/GRU for Sequence Prediction**: Text generation, machine translation.
- **Attention Mechanism and Transformers**: Advanced techniques for sequence-to-sequence tasks.

---

6. Neural Networks for Computer Vision

6.1 Image Classification

- **CNNs for Object Recognition**: Techniques to classify and label objects in images.
- **Fine-tuning Pre-trained Models**: Using models like VGG, ResNet, Inception for custom tasks.

6.2 Object Detection and Segmentation

- **Object Detection Models**: YOLO (You Only Look Once), Faster R-CNN, RetinaNet.
- **Semantic Segmentation**: U-Net, FCN (Fully Convolutional Networks).

6.3 Transfer Learning in Computer Vision

- **What is Transfer Learning?**: Using pre-trained models on large datasets (like ImageNet) for new tasks.
- **Fine-tuning a Pre-trained Model**: Adjusting the last layers of a pre-trained model for your specific task.

---

7. Neural Networks for Time Series and Forecasting

7.1 Sequence Prediction

- **RNNs for Sequence Modeling**: Predicting future data points in a sequence.

- **LSTMs for Long-Term Dependencies**: Capturing long-term patterns in data sequences.

7.2 Time Series Forecasting

- **Supervised Learning for Forecasting**: Using past observations to predict future values.
- **Applications**: Stock price prediction, weather forecasting.

---

8. Advanced Topics and Research in Neural Networks

8.1 Deep Reinforcement Learning

- **Reinforcement Learning Basics**: Agent, environment, rewards, policies.
- **Deep Q Networks (DQN)**: Combining deep learning with Q-learning.
- **Applications of Deep RL**: Robotics, game playing (e.g., AlphaGo, Atari).

8.2 Neural Architecture Search (NAS)

- **What is NAS?**: Automating the process of designing neural network architectures.
- **Search Techniques**: Evolutionary algorithms, reinforcement learning-based search.

8.3 Neural Networks in Healthcare

- **Medical Image Analysis**: Using CNNs for disease detection (e.g., tumor detection in MRI scans).
- **Predicting Patient Outcomes**: Using LSTMs or other architectures for time-series analysis in medical data.

---

9. Implementing Neural Networks in Practice

9.1 Building Neural Networks with Frameworks

- **TensorFlow**: Creating neural networks with TensorFlow and Keras.
    - Setting up a neural network architecture.
    - Training the model, evaluating performance, and hyperparameter tuning.
- **PyTorch**: Implementing neural networks in PyTorch with dynamic computation graphs.
    - Building and training models in PyTorch.
- **Keras**: High-level interface for building neural networks (integrated with TensorFlow).

9.2 Model Evaluation and Hyperparameter Tuning

- **Metrics for Evaluation**: Accuracy, precision, recall, F1-score, AUC-ROC.
- **Hyperparameter Tuning**: Grid search, random search, and Bayesian optimization.
- **Cross-Validation**: Ensuring model generalization using k-fold cross-validation.

---

10. Neural Network Deployment and Monitoring

10.1 Model Deployment

- **Serving Models**: Using Flask, FastAPI, or TensorFlow Serving for real-time predictions.
- **Deployment Platforms**: Deploying models to cloud services like AWS, GCP, or Azure.

10.2 Monitoring and Maintenance

- **Model Drift**: Detecting when a model's performance decreases over time due to changes in the data distribution.
- **Continuous Monitoring**: Tools for monitoring model performance in production.

Syllabus Summary:

1. **Introduction to Neural Networks**: Basics, components, and terminology.
2. **Building Simple Neural Networks**: Perceptron, MLP, and training algorithms.
3. **Optimization and Learning**: Gradient Descent, regularization, and optimizers.
4. **Advanced Architectures**: CNNs, RNNs, GANs, Transformers, Autoencoders.
5. **Neural Networks for NLP and CV**: Word embeddings, RNNs, object detection.
6. **Time Series and Forecasting**: Using RNNs for sequence prediction.
7. **Research Topics**: Reinforcement Learning, Neural Architecture Search, Healthcare.
8. **Practical Implementation**: Frameworks (TensorFlow, PyTorch, Keras), hyperparameter tuning.
9. **Deployment**: Serving models in production and monitoring.